

AMIHACKS 2026

Three Problem Statements: Approach, Stack & Build Plan

Team

24-Hour Hackathon

September 24, 2026

Three Tracks — One Slide

Track A — NGO

Surplus-to-Shelter

Real-Time Food Rescue Routing

Online bipartite
assignment
+ routing

Track B — Open Innovation

CityPulse

Live Civic Health Dashboard

Stream fusion +
anomaly detection

Track C — Deep-Tech

SentinelAPI

Zero-Trust API Scanner

Differential program
testing

Track A — Problem Framing

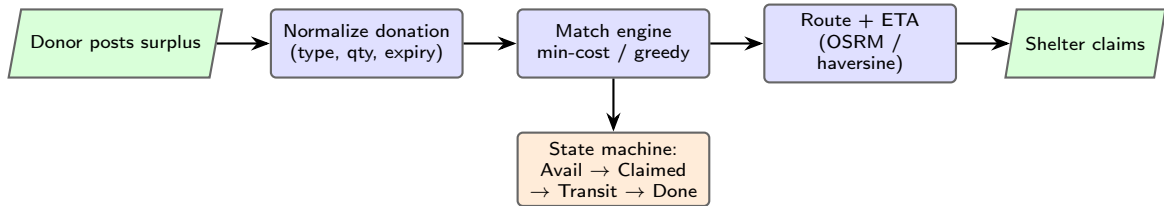
Not a CRUD app — an **online bipartite assignment** problem over a spatio-temporal stream:

- Donations arrive dynamically: (*location, food_type, qty, expiry, cold_chain*)
- Shelters = resources with capacity & compatibility constraints
- Every donation has a **hard deadline** (expiry) $\Rightarrow$ deadline-constrained assignment with travel cost

Objective

Minimize wasted food: match each donation to a compatible shelter that can receive it before expiry, at minimum road-network cost.

Track A — Flow



Track A — Matching Code

```
1 import numpy as np
2 from scipy.optimize import linear_sum_assignment
3
4 def match(donors, shelters):
5     n, m = len(donors), len(shelters)
6     cost = np.full((n, m), np.inf)          # inf = incompatible
7     for i, d in enumerate(donors):
8         for j, s in enumerate(shelters):
9             if compatible(d, s):             # type + cold-chain + capacity
10                 cost[i, j] = road_distance(d, s)
11     rows, cols = linear_sum_assignment(cost)  # O(n^3) min-cost
12     return [(donors[i], shelters[c]) for i, c in zip(rows, cols)
13             if cost[i, c] < np.inf]
```

MVP: greedy + deadline heap. Ceiling: OR-Tools CP-SAT (routing + assignment).

Tech stack

- Backend: FastAPI (async)
- Matching: SciPy → OR-Tools
- Roads: haversine → OSRM
- DB: SQLite → PostGIS
- Realtime: WebSocket / SSE
- UI: Next.js + MapLibre GL

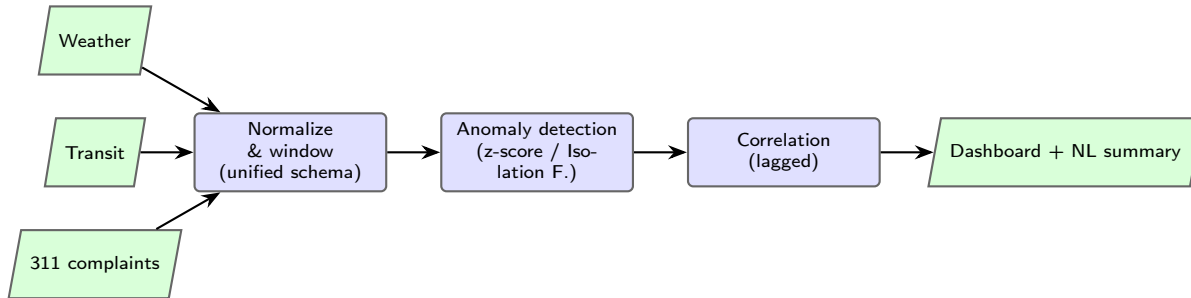
Build steps

- 1 Donation + shelter models
- 2 Compatibility & distance fns
- 3 Min-cost matcher
- 4 State machine + WebSocket
- 5 Live map UI
- 6 Demo: post → match → ETA

Stream fusion + online anomaly detection — not a dashboard problem.

- Feeds differ in **rate** (weather hourly, transit per-minute, 311 bursty), **format**, and **identity** (no common place_id)
- Canonical event model:
(event_type, t, geohash, severity, source)
- Correlation must be **honest**: co-occurrence $\neq$ causation — label links as “possibly related”

Track B — Flow



Track B — Anomaly Detection Code

```
1 from collections import deque
2
3 W = deque(maxlen=1440)                # rolling window
4
5 def flag(event):
6     W.append(event.value)
7     mu = sum(W) / len(W)
8     var = sum((x - mu) ** 2 for x in W) / len(W)
9     sd = var ** 0.5
10    z = (event.value - mu) / (sd + 1e-9)
11    return z > 3.0                    # 3-sigma anomaly
```

Ceiling: Isolation Forest / STL decomposition / transformer autoencoder.

Track B — Stack & Steps

Tech stack

- Ingestion: asyncio pollers → Kafka/Bytewax
- Store: Postgres → TimescaleDB
- Anomaly: z-score → Isolation Forest
- Summary: template NLG → grounded LLM
- UI: Next.js + MapLibre + WebSocket
- Replay: seeded synthetic dataset

Build steps

- 1 Simulate 3 feeds
- 2 Normalize to one schema
- 3 Rolling-window anomaly
- 4 Lagged correlation
- 5 Plain-language summary
- 6 Live map + replay

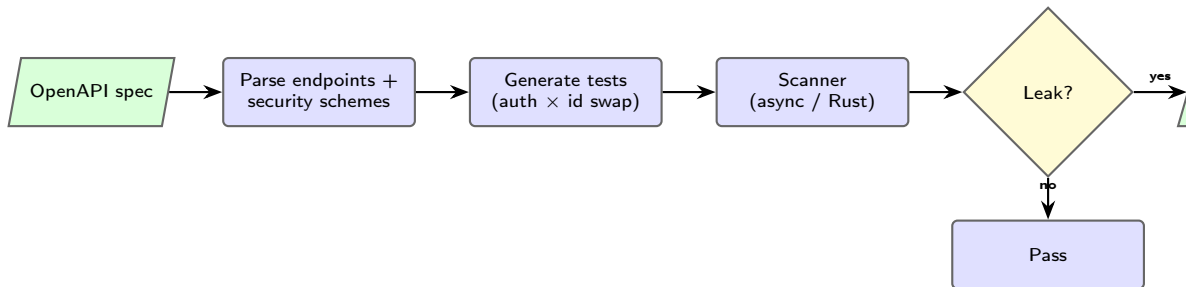
Differential program testing against a spec — not blind fuzzing.

- Compare **behaviour across security contexts** and flag divergence from the spec's contract
- The oracle = cross-account response diff + schema-vs-actual-keys diff
- **Key tactic**: build the sandbox API *and* the scanner, so precision is verifiable and the demo is deterministic

Classes implemented

(1) BOLA / IDOR (2) Excessive data exposure (3) stretch: broken auth / missing rate-limit

Track C — Flow



Track C — IDOR Differential Test

```
1 async def test_bola(client, endpoints):
2     a, b = await register(client), await register(client)    # two users
3     findings = []
4     for ep in endpoints_with_id(endpoints):
5         r = await client.get(ep.path.format(id=b.id),        # A's token, B's id
6                               headers=a.auth)
7         if r.json().get("user_id") == b.id:                  # cross-account leak
8             findings.append(Finding("CRITICAL", "BOLA", ep,
9                                     curl_repro(ep, a, b)))
10    return findings
```

Oracle: response returns a resource owned by a different principal.

Tech stack

- Sandbox API: FastAPI (auto OpenAPI)
- Engine: Python httpx → Rust (tokio + reqwest)
- Spec: openapi.json / Swagger 2.0
- Oracle: schema diff + cross-account diff
- Report: findings.json → dashboard
- Advanced: LLM test-gen, agentic chains

Build steps

- 1 Seed vulnerable API (IDOR + exposure)
- 2 Parse OpenAPI spec
- 3 Generate auth-context tests
- 4 Differential oracle
- 5 Severity ranking + curl repro
- 6 Dashboard

Comparative Summary

Criterion	A (Food)	B (CityPulse)	C (SentinelAPI)
Time-to-demo	Fastest	Medium	Medium
Finish risk	Low	Medium	Medium
Wow-factor	Medium	High	Highest
External deps	None	3+ feeds	None
Demo reliability	High	Medium	Highest
Novelty	Medium	Med-High	Highest

Recommendation

Safest = A. Best wow + still finishable = **C (SentinelAPI)** — deterministic demo, zero external dependencies, deep-tech story.

Thank you